1_arrays.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    // Time Complexity:
    // Read: O(1)
    // Update: O(1)
    // Traverse: O(n)
    // Find Maximum: O(n)
    // Insert: O(n)

    // Array with one extra space for insertion
    int marks[6] = {75, 82, 90, 68, 88};
    int size = 5; // Current number of elements

    // Original Array
    cout << "Original Array: ";
    for (int i = 0; i < size; i++) {
        cout << marks[i] << " ";
    }
    cout << endl;

    // 1. Read a value at a given index
    int index = 2;
    cout << "\n1. Read Operation" << endl;
    cout << "Value at index " << index << ": " << marks[index] << endl;

    // 2. Update a value at a given index
    int updateIndex = 3;
    int newValue = 80;
    marks[updateIndex] = newValue;

    cout << "\n2. Update Operation" << endl;
    cout << "Updated Array: ";
    for (int i = 0; i < size; i++) {
        cout << marks[i] << " ";
    }
    cout << endl;

    // 3. Traverse and print all values
```

```
    cout << "\n3. Traverse Operation" << endl;
    for (int i = 0; i < size; i++) {
        cout << "Index " << i << " = " << marks[i] << endl;
    }

    // 4. Find the maximum value
    int maximum = marks[0];
    for (int i = 1; i < size; i++) {
        if (marks[i] > maximum) {
            maximum = marks[i];
        }
    }

    cout << "\n4. Maximum Value: " << maximum << endl;

    // 5. Insert a value at a given index
    cout << "\n5. Insert Operation" << endl;

    cout << "Array Before Insertion: ";
    for (int i = 0; i < size; i++) {
        cout << marks[i] << " ";
    }
    cout << endl;

    int insertIndex = 2;
    int insertValue = 95;

    // Shift elements to the right
    for (int i = size; i > insertIndex; i--) {
        marks[i] = marks[i - 1];
    }

    // Insert new value
    marks[insertIndex] = insertValue;
    size++;

    cout << "Array After Insertion: ";
    for (int i = 0; i < size; i++) {
        cout << marks[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Output

Original Array: 75 82 90 68 88

1. Read Operation
Value at index 2: 90

2. Update Operation
Updated Array: 75 82 90 80 88

3. Traverse Operation
Index 0 = 75
Index 1 = 82
Index 2 = 90
Index 3 = 80
Index 4 = 88

4. Maximum Value: 90

5. Insert Operation
Array Before Insertion: 75 82 90 80 88
Array After Insertion: 75 82 95 90 80 88

---

1. Create the array

int marks[6] = {75, 82, 90, 68, 88};

- The array has **6 spaces**.
- Five spaces store marks.
- One extra space is reserved for insertion, exactly as required in the assignment.

2. Read

cout << marks[index];

Reads the value stored at a specific index.

Time Complexity: O(1)

3. Update

marks[updateIndex] = newValue;

Replaces the old value with a new one.

Time Complexity: O(1)

4. Traverse

for(int i = 0; i < size; i++)

Prints every element in the array.

Time Complexity: O(n)

5. Find Maximum

Compares all elements to determine the largest value.

Time Complexity: O(n)

6. Insert

```
for (int i = size; i > insertIndex; i--) {
    marks[i] = marks[i - 1];
}
```